

Implementación de MLP con Adam como optimizador para clasificación de trabajos del cluster Kabre

Proyecto 2 - MC-8851 Optimización y Programación Numérica

Daniel Garbanzo Luis Carlos Navarro Todd
Ricardo Shum Estrada

2026-06-19

Tabla de contenidos

1	Marco teórico	2
1.1	1. Función de pérdida	2
1.2	2. Funciones de activación	2
1.3	3. Momentos estadísticos en el contexto de machine learning	3
1.4	4. Redes neuronales y perceptrón multicapa	3
1.5	5. Backpropagation	3
1.6	6. Adam como optimizador	4
2	Referencias	5

Integrantes y carnes

Integrante	Carne
Daniel Garbanzo	2022117129
Luis Carlos Navarro Todd	2022212158
Ricardo Shum Estrada	2017144193

Algoritmo: MLP con Adam como optimizador

Dataset: Historial de trabajos del cluster Kabre de CeNAT/CONARE

Fuente del dataset: dataset.csv del repositorio Project-1-CeNAT

Entregable base: mlp_adam.ipynb

1. Marco teórico

1.1. 1. Función de pérdida

Una función de pérdida (*loss function*) cuantifica la diferencia entre la salida predicha por un modelo y el valor real esperado. En el contexto de machine learning, el entrenamiento de un modelo consiste en encontrar los parámetros θ que minimizan el valor promedio de la función de pérdida sobre el conjunto de entrenamiento (Goodfellow, Bengio, & Courville, 2016):

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i(\theta))$$

La elección de la función de pérdida depende de la naturaleza del problema. Para tareas de clasificación binaria, como la utilizada en este proyecto, la pérdida de entropía cruzada es la más utilizada, ya que penaliza con mayor fuerza las predicciones incorrectas:

$$L_{CE}(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

La función de pérdida no solo mide el desempeño del modelo; también define la superficie sobre la cual operan los algoritmos de optimización, de modo que su forma (convexa, no convexa, diferenciable) condiciona directamente la dificultad del proceso de entrenamiento (Bishop, 2006).

1.2. 2. Funciones de activación

Las funciones de activación introducen no linealidad en la salida de cada neurona artificial. Sin ellas, una red de múltiples capas se reduciría algebraicamente a una única transformación lineal, sin capacidad de modelar relaciones complejas entre las variables de entrada y la salida (Goodfellow et al., 2016).

Históricamente, la función sigmoide y la tangente hiperbólica fueron las primeras en utilizarse de forma extendida:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Ambas comprimen su entrada en un rango acotado, pero presentan gradientes muy pequeños cuando $|z|$ es grande, lo que dificulta el entrenamiento de redes profundas mediante el fenómeno conocido como desaparición del gradiente (*vanishing gradient*) (Glorot & Bengio, 2010). Como respuesta a esta limitación, se popularizó la unidad lineal rectificada (ReLU):

$$f(z) = \max(0, z)$$

La ReLU mantiene un gradiente constante para valores positivos de z , lo que acelera la convergencia del entrenamiento en arquitecturas profundas y reduce el costo computacional respecto a funciones exponenciales (Nair & Hinton, 2010). La selección de la función de activación afecta tanto la

expresividad del modelo como la estabilidad numérica del proceso de backpropagation descrito más adelante.

1.3. 3. Momentos estadísticos en el contexto de machine learning

Dentro del entrenamiento de redes neuronales, los momentos estadísticos ayudan a estimar de manera más eficiente los gradientes. En optimizadores adaptativos como Adam, se estiman en línea el primer momento (la media de los gradientes) y el segundo momento (la media de los gradientes al cuadrado, una medida de varianza no centrada) para ajustar dinámicamente la magnitud de cada actualización de los parámetros (Kingma & Ba, 2015). De esta manera, un concepto puramente estadístico se traduce en un mecanismo práctico de control de la velocidad y estabilidad del aprendizaje.

1.4. 4. Redes neuronales y perceptrón multicapa

El perceptrón, propuesto originalmente como un modelo simplificado de neurona biológica, calcula una combinación lineal de sus entradas seguida de una función de activación (Rosenblatt, 1958):

$$\hat{y} = f(w^T x + b)$$

Un perceptrón individual solo puede separar clases linealmente separables; Minsky y Papert (1969) demostraron formalmente que un único perceptrón no puede representar funciones tan simples como la disyunción exclusiva (XOR). Esta limitación motivó el desarrollo del perceptrón multicapa (*Multilayer Perceptron*, MLP), una red neuronal compuesta por una o más capas ocultas entre la entrada y la salida.

En un MLP, la activación de la capa l se obtiene a partir de la capa anterior mediante:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

donde $W^{(l)}$ y $b^{(l)}$ son la matriz de pesos y el vector de sesgos de la capa l , y f es la función de activación correspondiente. Apilar varias de estas transformaciones permite que la red aproxime funciones no lineales complejas. (Hornik, Stinchcombe, & White, 1989). Esta capacidad de composición es lo que distingue a las redes neuronales profundas de los modelos lineales clásicos (Haykin, 2009).

1.5. 5. Backpropagation

El algoritmo de retropropagación del error (*backpropagation*) permite calcular de manera eficiente el gradiente de la función de pérdida respecto a todos los parámetros de una red neuronal, aplicando sistemáticamente la regla de la cadena desde la capa de salida hacia las capas anteriores (Rumelhart, Hinton, & Williams, 1986).

Para la capa de salida L , el error se define como:

$$\delta^{(L)} = \nabla_a L \odot f'(z^{(L)})$$

donde $\odot$ denota el producto elemento a elemento. Para cualquier capa oculta l , el error se propaga hacia atrás utilizando los pesos de la capa siguiente:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)})$$

Una vez calculados los términos $\delta^{(l)}$, el gradiente respecto a los pesos de cada capa se obtiene directamente como:

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

Este procedimiento evita recalcular derivadas redundantes capa por capa, lo que reduce drásticamente el costo computacional frente a un cálculo numérico ingenuo o de fuerza bruta del gradiente, y es la base sobre la cual operan prácticamente todos los algoritmos modernos de entrenamiento de redes neuronales (Goodfellow et al., 2016; LeCun, Bengio, & Hinton, 2015).

1.6. 6. Adam como optimizador

Adam (*Adaptive Moment Estimation*) es un algoritmo de optimización basado en gradiente que combina dos ideas previas: el momentum, que acumula una media móvil de los gradientes para suavizar la trayectoria de descenso, y la adaptación de la tasa de aprendizaje por parámetro, inspirada en algoritmos como RMSProp (Kingma & Ba, 2015).

En cada iteración t , Adam actualiza dos estimadores exponenciales: el primer momento m_t (media de los gradientes) y el segundo momento v_t (media de los gradientes al cuadrado):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Dado que estos estimadores están sesgados hacia cero en las primeras iteraciones, Adam aplica una corrección de sesgo:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Finalmente, los parámetros se actualizan utilizando ambos momentos corregidos:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

donde η es la tasa de aprendizaje y ϵ es una constante pequeña que evita divisiones por cero. Los valores recomendados por los autores originales son $\beta_1 = 0.9$, $\beta_2 = 0.999$ y $\epsilon = 10^{-8}$ (Kingma & Ba, 2015). Gracias a esta combinación de momentum y escalamiento adaptativo, Adam suele converger más rápido y de forma más estable que el descenso de gradiente estocástico clásico en arquitecturas profundas, lo que explica su adopción generalizada como optimizador por defecto en la práctica moderna de deep learning (Goodfellow et al., 2016).

2. Referencias

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 9, 249-256.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- Haykin, S. (2009). *Neural networks and learning machines* (3rd ed.). Pearson.
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359-366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>
- Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807-814.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. <https://doi.org/10.1037/h0042519>
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>